

AI Engineer Intern: Take-Home Case Study

Building an Automated Sales-Call Intelligence System for FitNova

Role	AI Engineer Intern (6-month, in-office, Bangalore)
Format	Take-home assignment. A hands-on building task, not a slide deck.
Duration	48 hours
Submission	A single .zip with your code, a README, and a short writeup, plus a 2-minute video walkthrough.

Context

FitNova is a fast-growing, Bangalore-based online platform for personalized fitness and wellness coaching. People discover programs online, but enrollment still happens over the phone. A tele-advisor calls the lead, understands their goals and budget, books a free trial session (in-person or online), recommends a plan, and guides them to enroll. Choosing a coaching program runs on trust: people worry about wasted money, unrealistic promises, and pushy sales, so the quality of that advisor conversation directly drives both conversion and reputation.

You are not building this for one person. A Sales Director wants to see the health of the whole org. Team Leaders want to coach the advisors on their own team. Advisors want to understand and improve their own calls. Keep all of them in mind as you design.

Problem

Today, FitNova has hundreds of advisors organised into pods, each led by a Team Leader who rolls up to the Sales Director. Quality control is manual: a Team Leader listens to a handful of calls per advisor each week and coaches from memory. It does not scale, it is inconsistent between leaders, and most calls are never reviewed, so mis-selling, weak discovery, and missed follow-ups go unnoticed until a customer complains.

The opportunity is an AI system that automatically transcribes every sales call, separates who said what, scores call quality, flags issues, and surfaces it for the people above, with a way for advisors to contest unfair flags. You are the AI Engineer intern brought in to build it end to end: find the manual work, and replace it with automations, tools, and agents that save the team hours every week.

A System Design

Map the complete pipeline that turns a raw sales call into reviewed, actionable insight. Walk through:

- **Ingestion:** getting call recordings and metadata out of the call source (telephony, dialer, CRM export, or a folder of files).
- **Transcription and diarisation:** turning audio into text and separating Advisor from Customer.
- **Analysis:** scoring call quality and flagging issues.
- **Storage:** where transcripts, tags, scores, and metadata live.
- **Surfacing:** the dashboards.

- **Feedback:** how humans correct the system and how that loops back.

Then call out the stages where automation adds the most value, and justify your prioritisation. A diagram (for example Excalidraw or Mermaid) plus a short written walkthrough is ideal. Design the ingestion layer to be source-agnostic: FitNova may switch telephony or CRM vendors, or run several at once. Build the system from scratch and choose tools as you see fit.

B Analysis Engine

Define how a call gets judged.

- **Scoring rubric:** the dimensions you score (for example needs discovery, product knowledge, objection handling, compliance, and next-step booking) and how they roll up into a per-call quality score and an org, team, and advisor average.
- **Issue-tag taxonomy:** the flags the system raises, each with a severity. Consider what hurts a coaching sale: no needs discovery, over-promising ("guaranteed results"), pressure or urgency tactics, price before value, undisclosed costs, weak or missing trial booking, and talking over the customer.
- **Reliable tagging:** your prompt or agent design, how you get structured and consistent output, and how you stop the model inventing flags. Each flag carries a timestamp, the quoted line, and a short reason.

Edge cases: account for the realistic ways a call breaks the pipeline, and how you handle each: mono recordings or poor diarisation, heavy Hindi-English code-switching or other languages, non-sales calls (wrong number, internal), PII that must be redacted, hallucinated or false-positive tags, and vendor API failures, with retries and idempotency so a call is never double-processed.

C Data & Storage

Define the data the system consumes and where it comes from, and the output it produces after each call. You choose the input source and the output format, but design ingestion to be source-agnostic, so a new telephony, dialer, CRM, or file source can be mapped in without code changes, and make the output rich enough to be useful downstream. Decide where transcripts, tags, scores, and metadata live.

Model the org structure too: an org has many teams, each team has many advisors, and it will grow. Design the data model so new teams and advisors can be added without manual reconfiguration.

Submission Guidelines

This is the main event: hand us a **working prototype**, not a description of one. We will run your code and may ask you to explain any part of it, so build what you understand.

Minimum expectation: a working system that does the job, running the full loop end to end. A call comes in, gets transcribed, diarised, scored, and flagged, and the result is stored and surfaced.

Submit two things:

- **Code (.zip):** a single .zip with your code, a README (setup, how to run, and what is real versus mocked), and your writeup. The demo must run, from one clear command or a deployed link.
- **Video walkthrough (required):** a short video, around 2 minutes, explaining your thinking in your own words, the trade-offs you made, what you chose not to build and why, and where the system would fail. Make sure the link is accessible and not restricted.

Brownie points: a genuinely well-built system with good visualization, dashboards that are clear and useful, and a polished, reliable end-to-end experience.

We value conciseness. Say more with less.

Evaluation

Two things matter most: your **depth of understanding** and a **working prototype**. Concretely, we grade on:

Dimension	What we look for
Completeness of thought	You cover the whole picture, not just the happy path: the pipeline end to end, the edge cases, and the data.
Depth of understanding	You understand the problem and your own choices, and can defend the trade-offs you made rather than name tools.
Working prototype	Something real runs: at least one call goes through the full loop end to end, with the result stored in a real database and surfaced.

There are no perfect answers. Show depth of understanding and ship something real. Good luck.